

8교시: 운영 플랫폼 연결과 구름 EXP 배움일기

장애 리포트

개발팀 전달용 인시던트 보고서

1 재현 조건 (Reproduction Steps)

1. 사용자 A로 로그인

2. 상품 X를 장바구니에 담기 (SKU: X-123)

3. 배송지 입력 후 주문하기 클릭

4. 간헐적으로 주문 생성 실패 발생

발생 시간: 2025-06-02 14:50 ~ 15:10 (KST)

재현 빈도: 약 30% (10회 중 3회 실패)

사용자 영향: 주문 생성 실패, 장바구니 유지

환경: Staging (stg-us-east-1)

2 관찰 결과 (Observed Behavior)

✓ 사용자에게 "주문 처리 중 오류가 발생했습니다." 메시지 노출

✓ 주문이 생성되지 않음

✓ 결제는 발생하지 않음

✓ 재시도 시 일부 성공

에러 비율: 28.7%

평균 응답 시간: 2.8s (성공) / 5.6s (실패)

영향 지표: 주문 생성 수 급감 (~62%)

3 의심 지점 (Suspected Boundary)

API Gateway → Order Service → Payment Service → Inventory Service

의심 지점: Order Service → Payment Service 호출 구간

Order Service 로그에서 외부 호출 이후 타임아웃 발생

Payment Service에서는 요청 수신 기록이 일부만 존재

네트워크/타임아웃 또는 계약 불일치 가능성

개발팀 Handoff

★ 핵심 요약

주문 생성 중 Order Service → Payment Service 호출에서 간헐적 실패 발생

사용자 영향: 주문 생성 실패 증가

의심 원인: 타임아웃 / 계약 불일치 / 예외 처리 누락

필요 조치 (개발팀)

Payment Service 호출 계약 및 타임아웃 설정 검토

예외/에러 처리 로직 보강 (재시도/폴백 포함)

역동성 보장 및 중복 방지 확인

필요 시 상세 에러 코드 및 메시지 개선

재확인 요건

수정 후 아래 재확인 시나리오로 검증 부탁드립니다.

1. 재현 조건 동일하게 10회 실행

2. 주문 생성 성공률 100% 확인

3. 에러 로그 및 알람 미발생 확인

연락 및 참고

문의: #ops-alerts (Slack)

대시보드: Dashboard > Order Service

관련 티켓: OPS-2025-0602-INC-012

4 로그 증거 (Log Evidence)

Order Service 로그 (일부)

2025-06-02T14:53:21.123Z ERROR [order-service] Failed to call PaymentService java.net.SocketTimeoutException: Read timed out at feign.Client.execute(Client.java:...) at ... traceId=abc123, orderId=78910

Payment Service 로그 (일부)

2025-06-02T14:53:21.987Z INFO [payment-service] POST /api/payments status=200 latency=120ms traceId=def456 ... (해당 시간대 일부 요청만 수신 기록 존재)

※ 전체 로그 및 트레이스는 첨부 (order-service.log, payment-service.log)

5 수정 요청 (Required Developer Change)

타임아웃 및 재시도 정책

Payment Service 호출 타임아웃 증가 또는 적절한 재시도(지수 백오프) 적용

에러 처리 개선

타임아웃/5xx 발생 시 사용자 친화적 에러 처리 및 폴백 로직(주문 일시 저장 등) 고려

계약 검증

요청/응답 스키마 및 필수 필드 재확인, 버전 호환성 검토

관측성 강화

에러 코드 세분화, 로그에 외부 호출 응답 코드/지연시간 추가

6 재확인 (Recheck Result)

재확인 일자: _____ 확인자: _____

항목	기준	결과	비고
주문 생성 성공률	100% (10/10)	☒ 통과 ☐ 실패	
에러 로그	발생 없음	☒ 통과 ☐ 실패	
알람	발생 없음	☒ 통과 ☐ 실패	
응답 시간 (p95)	≤ 2.0s	☒ 통과 ☐ 실패	

결론:

☒ 정상

☐ 추가 조치 필요 (_____)

특이사항:

수업 목표

- Day 2 사고 사례를 Kubernetes와 observability 필요성으로 연결한다.
- "Kubernetes가 해결하는 문제"와 "application이 해결해야 하는 문제"를 구분한다.
- 구름 EXP 배움일기를 사고 evidence 중심으로 작성한다.

오늘 본 사고와 연결 개념

사고	운영 플랫폼이 도와주는 것	application이 해결해야 하는 것
Ghost pending order	job 실행, metrics, alert	outbox, 보상 transaction
Worker backlog	replica scale, HPA	idempotent worker, 처리량 설계
Poison message	logs/metrics alert	DLQ, schema validation, retry policy
Duplicate request	traffic routing은 가능	idempotency key, unique constraint
Readiness gap	readinessProbe	business consistency

Kubernetes는 중요하지만 모든 것을 해결하지 않는다.

Kubernetes solves scheduling/restart/discovery/scaling primitives.

Application design solves consistency/idempotency/business recovery.

file:///mnt/c/Users/sunny/AppData/Local/Temp/lecture-pdf-lu12ndk/index.html

1/4

사고별 Kubernetes 연결

Day 2 evidence	Kubernetes/운영 개념
worker가 멈춤	Deployment, ReplicaSet
queue backlog 증가	metrics, HPA, alert
ready 아닌 service	readinessProbe
process hang	livenessProbe
service name 통신	Service DNS
환경별 설정	ConfigMap, Secret
사고 로그 분산	centralized logging, tracing
audit timeline	observability + business audit

하지만 K8s만으로 안 되는 것

문제	이유
DB commit 후 Redis publish 실패	transaction boundary 문제
중복 주문	idempotency 설계 필요
poison message 유실	DLQ/retry metadata 필요
업무 상태 reconciliation	application/job 설계 필요
결제/주문 보상 처리	domain policy 필요

이 구분을 못 하면 Kubernetes를 배우고도 운영 사고를 제대로 해결하지 못한다.

구름 EXP 배움일기 가이드

감상이 아니라 사고 분석 중심으로 쓴다.

항목	작성 가이드
선택한 사고	ghost pending, backlog, poison, duplicate 중 하나
재현 명령	실행한 script
client symptom	사용자가 봤을 결과
internal evidence	DB/queue/log/audit 중 핵심
mismatch	외부 결과와 내부 상태가 어떻게 달랐는가
immediate action	당장 할 조치
long-term fix	설계 보완
Kubernetes question	어떤 K8s 개념이 도움 되는가
application question	K8s로 해결 안 되는 것은 무엇인가

배움일기 예시

W3D2 배움일기

오늘은 ghost pending order 사고가 가장 인상적이었다.
 `01\_ghost\_pending\_order.sh`를 실행하자 client는 실패 응답을 봤지만,
 DB `orders` table에는 pending 주문이 남아 있었다.
 Redis queue에는 처리 event가 없어서 order-worker가 이 주문을 자동 처리할 수 없었다.

이 사고는 단순히 Redis를 다시 켜고 해결되는 문제가 아니라,
 DB commit과 queue publish 사이의 transaction boundary 문제라는 점을 알게 되었다.
 Kubernetes readinessProbe는 준비 안 된 pod로 traffic을 보내지 않는 데 도움을 주지만,
 outbox pattern이나 idempotency는 application에서 설계해야 한다.

다음 수업으로 넘길 질문

질문	다음 연결
worker replica를 늘리면 backlog가 줄어드는가	Deployment scale
readinessProbe는 어디까지 확인해야 하는가	probe 설계
Secret과 ConfigMap은 사고를 줄이는가	설정 관리
logs만으로 충분한가	metrics/tracing
Kubernetes Job으로 ghost pending을 보정할 수 있나	batch/reconciliation

핵심 포인트

Day 2의 결론:

D1: MSA 정상 경로를 실행했다.

D2: 정상 경로가 깨질 때 내부 상태가 어떻게 어긋나는지 봤다.

K8s: 이 운영 문제 중 일부를 플랫폼 수준에서 다루기 시작한다.

Evidence Note

W3D2S8 Learning Journal

- selected incident:
- script:
- client symptom:
- internal evidence:
- mismatch:
- Kubernetes helps with:
- application must solve: